

PROMPT PROTOKOLL: FOOD4STUDENTS

Gruppe 2 (Christov Alexander, Mihailescu David, Stella Hambuch)

Datum: 14.09. (Exercise 1)

AI: Google stitch

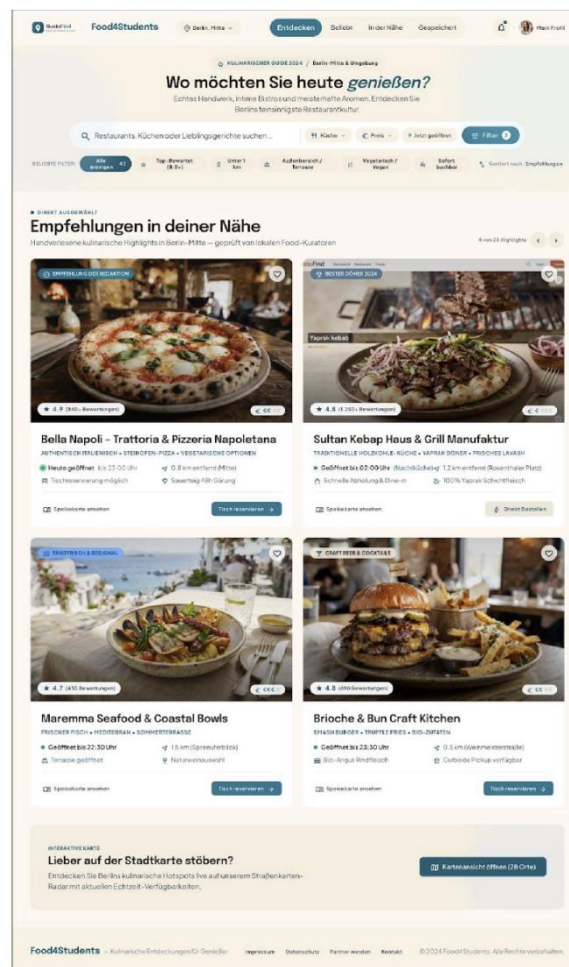
Prompt 1:

Wir brauchen ein Design für eine Restaurant-Finder-App. Es soll eine Suchleiste oben aufweisen, rechts davon Filtereinstellungen. Darunter sollen einige Empfehlungen mit Namen, Bild, Rating und Öffnungszeiten (als Bsp. ein Pizza-Restaurant und ein Kebap-Laden). Color Palette sollte Beige und Meeresblau sein, aber Meeresblau eher für Akzente und Details. Schriftgröße soll anpassend sein.

Prompt 2:

Kannst du den Namen von GustoFinder auf Food4students umändern?

Output:



Reflexion:

Positiv:

Wir sind sehr zufrieden mit dem Output. Die Farben wurden gut umgesetzt, und nach dem zweiten Prompt wurde der Name der Seite geändert. Auch die übrigen Vorgaben, also die Suchleiste und die Darstellung der Restaurants mit Name und Bild, wurden wie gewünscht umgesetzt.

Negativ:

Google Stitch hat mehr gemacht, als verlangt war. Zum Beispiel wurde eine Navigationsleiste oben mit Punkten wie „Entdecken“ und „In der Nähe“ ergänzt. Außerdem ist rechts oben ein Profil-Icon zu sehen, man soll sich also auf der Seite anmelden können.

Das muss man nicht zwingend negativ sehen, da wir dadurch weitere Ideen für unser Projekt bekommen haben. Trotzdem war es im Prompt nicht verlangt.

Datum: 24.09.(Exercise Data Part 2):

Prompt:

Marke;Typ;Preis;Farbe;Masse

- Welche Attribute gibt es noch bei Fahrrädern
- Gib mir 10 verschiedene Datensätze zu diesem Thema bzw. mach mir das in einer csv Datei

CSV-Code:

Preview	Code	Blame	11 lines (11 loc) · 484 Bytes
1	ID;Marke;Typ;Preis;Farbe		
2	1;Cube;Trekkingrad;799,00 €;Schwarz		
3	2;Specialized;Rennrad;5.990,00 €;Carbon / Rot		
4	3;Giant;E-Mountainbike;3.299,00 €;Lila (Purple Storm)		
5	4;Trek;Gravelbike;1.899,00 €;Olivgrün		
6	5;Canyon;Citybike;1.299,00 €;Grau		
7	6;Orbea;Mountainbike (Hardtail);999,00 €;Blau / Weiß		
8	7;KTM;E-Trekkingrad;3.499,00 €;Dunkelblau		
9	8;Scott;Kinderfahrrad;429,00 €;Gelb (Mimosa Yellow)		
10	9;Bulls;Jugend-MTB;649,00 €;Rot / Schwarz		
11	10;Woom;Kinderrad;449,00 €;Rot		

CSV Proof:

```
Debug Console  Problems  Output  Ports  Terminal

C:\Users\alexa\Desktop\Übung 2 CSV>node read.js
Fehler beim Lesen der Datei: fahrrad is not defined

C:\Users\alexa\Desktop\Übung 2 CSV>node read.js
--- Inhalt der Datei: fahrrad.csv ---

ID           | Marke           | Typ           | Preis          | Farbe
1            | Cube            | Trekkingrad   | 799,00 €       | Schwarz
2            | Specialized     | Rennrad       | 5.990,00 €     | Carbon / Rot
3            | Giant           | E-Mountainbike| 3.299,00 €     | Lila (Purple Storm)
4            | Trek            | Gravelbike    | 1.899,00 €     | Olivgrün
5            | Canyon          | Citybike      | 1.299,00 €     | Grau
6            | Orbea           | Mountainbike (Hardtail)| 999,00 €       | Blau / Weiß
7            | KTM             | E-Trekkingrad | 3.499,00 €     | Dunkelblau
8            | Scott           | Kinderfahrrad | 429,00 €       | Gelb (Mimosa Yellow)
9            | Bulls           | Jugend-MTB    | 649,00 €       | Rot / Schwarz
10           | Woom            | Kinderrad     | 449,00 €       | Rot

C:\Users\alexa\Desktop\Übung 2 CSV>
```

Datum: 10.01 (Exercise 3 Part 1)

AI: Claude Sonnet 5.5

Prompt 1:

Erstelle mir aus folgender Beschreibung einen Algorithmus in der Sprache JavaScript. Wir brauchen einen Algorithmus, der die Anforderungen des Benutzers mit den verschiedenen gespeicherten Daten von allen Restaurants der Datenbank vergleicht und dann das bestpassende Restaurant empfiehlt.

Jedes Restaurant könnte einen Matching-Score bekommen: Maximale Punkte insgesamt sind 100! Der Benutzer gibt die Gewichtung vor, indem er für jedes Merkmal eine Zahl eingibt zwischen 0 und 10, die beschreibt, wie wichtig dieses Merkmal für die Filterung (Algorithmus) sein soll. Die Bewertungen sollten individuell umgerechnet werden mit folgender Formel.

Bsp.: Gastronomie 6, Preis 5, Bezirk 4. Die Bewertungen sollten zusammengerechnet werden, im Bsp. also 15. Wir rechnen dann für Gastronomie $6/15 * 100 = 40$ Punkte und so weiter.

Dann: Restaurant A wird empfohlen, weil es am meisten Punkte aufweist.

Restaurant A = 80 Punkte

Restaurant B = 60 Punkte

Restaurant C = 40 Punkte

Input: Der Benutzer wählt Küche (Liste), maximalen Preis (Slider - \$ - \$\$ - \$\$\$) und Bezirk (Liste) und kann danach mit Slidern einstellen, wie wichtig diese Eingaben für den Benutzer sind.

Output: Die Restaurants sollten nach Punkten sortiert werden, das Restaurant, das am meisten Punkte hat, sollte ganz oben sein und danach die restlichen auch nach Punkteanzahl gereiht. Die Punkte sollten zu jedem Restaurant angezeigt werden (87/100). Als Platzhalter erstelle 3 italienische und 3 chinesische Restaurants, die sich im 20. Bezirk befinden. Gib jedem Restaurant eine andere Preisklasse.

Prompt 2:

Okay, kannst du mir die Eingabe als Benutzer auch ermöglichen?

Prompt 3:

Okay, wie würde ich die zwei Programme in Visual Studio Code verknüpfen?

The screenshot shows a web application interface for restaurant recommendations. It is divided into three main sections. The top section contains three input fields: a dropdown menu for 'Küche' (Cuisine) with 'Italienisch' (Italian) selected, a slider for 'Maximaler Preis: \$\$' (Maximum Price) set to the second position, and a dropdown menu for 'Bezirk' (District) with '20. Bezirk' (20th District) selected. The middle section is titled 'Wie wichtig ist dir das jeweilige Merkmal? (0 = egal, 10 = sehr wichtig)' (How important is each feature to you?). It contains three sliders: 'Küche: 7' (Cuisine) set to 7, 'Preis: 5' (Price) set to 5, and 'Bezirk: 10' (District) set to 10. The bottom section displays a recommendation for 'Trattoria Brigittenau', which is Italian, in the 20th district, with a score of '100/100' and a red 'Empfehlung' (Recommendation) badge.

Reflexion:

Positiv:

Die KI hat im Großen und Ganzen genau das geliefert, was wir uns vorgestellt hatten. Als ersten Schritt wollten wir ein Eingabefeld, in dem man Küche, Preis und Bezirk auswählen kann. Anschließend sollte man mit Slidern festlegen können, wie wichtig diese Angaben für die Empfehlung sind.

Negativ:

Unerwartet war nur ein Punkt: Wenn man „Italienisch“ als Küche auswählt und diese als weniger wichtig bewertet, werden Restaurants mit einer anderen Küche, zum Beispiel Chinesisch, teilweise vor italienischen Restaurants angezeigt. Wir hätten uns gewünscht, dass die ausgewählte Küche immer Vorrang hat und italienische Restaurants trotzdem zuerst erscheinen.

Nachtrag 07.10.:

Wir haben das nicht mehr geändert, da die Slider genau dafür gedacht sind, dass Nutzer:innen ihre Prioritäten selbst festlegen. Wenn jemandem die Küche weniger wichtig ist, dafür aber der Preis umso mehr (was bei Studierenden häufig vorkommt :D), ist die Ausgabe grundsätzlich richtig.

Datum: 05.10. (Finalizing our project)

Prompt 1: Kannst du uns im folgenden Format für den Bezirk 5 Restaurants erstellen: (Format im Code)

The screenshot shows a chat window with Claude AI. The prompt asks for 5 restaurants in a specific JSON format for a given district. The response is a JSON array of 15 restaurant objects, grouped by district (19, 20, 21, 22, 23). Each object contains name, cuisine, price, and district.

Prompt:

```
Kannst du uns in folgenden format für jeden Bezirk 5 restaurants erstellen: const restaurants = [ { name: "Trattoria Briggittenau", kueche: "Italienisch", preis: "$", bezirk: 20 }, { name: "Osteria Augarten", kueche: "Italienisch", preis: "$$", bezirk: 20 }, { name: "Ristorante Donaublick", kueche: "Italienisch", preis: "$$$", bezirk: 20 }, { name: "Goldener Drache", kueche: "Chinesisch", preis: "$", bezirk: 20 }, { name: "Jadegarten", kueche: "Chinesisch", preis: "$$", bezirk: 20 }, { name: "Kaiserpalast", kueche: "Chinesisch", preis: "$$$", bezirk: 20 }, ];
```

Response (Restaurants.js):

```
[ { name: "Osteria Grinzing", kueche: "Italienisch", preis: "$$", bezirk: 19 }, { name: "Ristorante Grinzing", kueche: "Italienisch", preis: "$$$", bezirk: 19 }, { name: "Lotus Grinzing", kueche: "Chinesisch", preis: "$$", bezirk: 19 }, { name: "Bambus Grinzing", kueche: "Chinesisch", preis: "$$$", bezirk: 19 }, // 20. Bezirk { name: "Trattoria Briggittenau", kueche: "Italienisch", preis: "$", bezirk: 20 }, { name: "Osteria Augarten", kueche: "Italienisch", preis: "$$", bezirk: 20 }, { name: "Ristorante Donaublick", kueche: "Italienisch", preis: "$$$", bezirk: 20 }, { name: "Goldener Drache", kueche: "Chinesisch", preis: "$", bezirk: 20 }, { name: "Jadegarten", kueche: "Chinesisch", preis: "$$", bezirk: 20 }, // 21. Bezirk { name: "Trattoria Floridsdorf", kueche: "Italienisch", preis: "$", bezirk: 21 }, { name: "Osteria Floridsdorf", kueche: "Italienisch", preis: "$$", bezirk: 21 }, { name: "Ristorante Floridsdorf", kueche: "Italienisch", preis: "$$$", bezirk: 21 }, { name: "Goldener Drache Floridsdorf", kueche: "Chinesisch", preis: "$", bezirk: 21 }, { name: "Peking Floridsdorf", kueche: "Chinesisch", preis: "$$", bezirk: 21 }, // 22. Bezirk { name: "Trattoria Donaue", kueche: "Italienisch", preis: "$", bezirk: 22 }, { name: "Osteria Donaue", kueche: "Italienisch", preis: "$$", bezirk: 22 }, { name: "Ristorante Donaue", kueche: "Italienisch", preis: "$$$", bezirk: 22 }, { name: "Bambus Donaue", kueche: "Chinesisch", preis: "$$", bezirk: 22 }, { name: "Lotus Donaue", kueche: "Chinesisch", preis: "$$$", bezirk: 22 }, // 23. Bezirk { name: "Trattoria Liesing", kueche: "Italienisch", preis: "$", bezirk: 23 }, { name: "Osteria Liesing", kueche: "Italienisch", preis: "$$", bezirk: 23 }, { name: "Ristorante Liesing", kueche: "Italienisch", preis: "$$$", bezirk: 23 }, { name: "Kaiserpalast Liesing", kueche: "Chinesisch", preis: "$", bezirk: 23 }, { name: "Jadegarten Liesing", kueche: "Chinesisch", preis: "$$$", bezirk: 23 }, ];
```

Output:

110 Elemente. Diese haben wir unverändert in den restaurants.js-Code eingefügt.

Prompt 2:

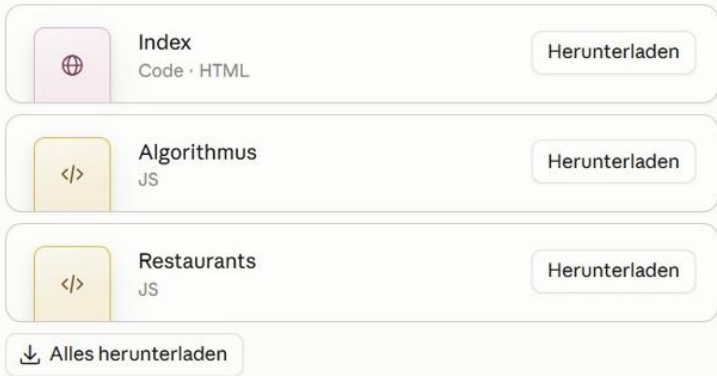
Wir müssen unsere HTML-Website schöner machen. Verwende dieses Bild als Inspiration (Exercise 1: Design von Google Stitch). Logo soll den Food4Students-Text haben und ein rundes Drop-Pin-Logo sein. Man soll auf der Search-Bar klicken können, um das jetzige Menü aufmachen zu können und die Küche-, Preis- etc. Einstellungen eingeben zu können. Verwende richtige Images für die Restaurants, die angegeben werden im Bild. Nach der Eingabe von Präferenzen sollen die Restaurants sich updaten, basierend auf der richtigen Aussage.

Output:

Bilder einfügen
Lege Fotos in den Ordner `images/` mit diesen Namen: `italienisch-1.jpg`, `italienisch-2.jpg`, `italienisch-3.jpg`, `chinesisch-1.jpg` und `chinesisch-2.jpg`. Kostenlose Fotos gibt es auf unsplash.com und pexels.com. Fehlt ein Bild, zeigt die Karte einen Farbverlauf mit Emoji.

Dateien im Ordner `restaurant-app/`: `index.html`, `algorithmus.js` und `restaurants.js` (die 115 Restaurants). Das alte Restaurant-Array steckt nicht mehr in `algorithmus.js`.

Die Restaurants aus deinem Bild (Döner, Seafood, Burger) gibt es in unseren Daten nicht, weil wir nur Italienisch und Chinesisch haben. Wenn ihr diese Küchen auch wollt, ergänze ich die Daten und Bilderlisten.



↓ Alles herunterladen

- 3 Files (Index, Algorithmus, Restaurants)
- Bei Visual Studio eingefügt

Prompt 3: Kannst du die CSS-Dateien aus der HTML-Datei herausfiltern und in eine separates CSS Datei eingeben?

Datum: 06.10. (Finalizing our project)

Reflexion:

Wir haben uns Task 1 nochmal durchgelesen und dabei festgestellt, dass wir viele Requirements, die wir uns vorgenommen hatten, nicht umgesetzt haben, dafür aber andere eingebaut haben.

Grundsätzlich haben wir viele Funktionen. Heute haben wir aber noch 1–2 Änderungen vorgenommen, um die geplanten Requirements, die sich noch umsetzen lassen, mit aufzunehmen.

Prompt 1:

Okay, das passt alles schon mal, aber ich hätte gerne eine Änderung. Wir wollen, dass man auf die Restaurants klicken kann, um ein Detail-Panel aufzumachen, dieses Panel zeigt alle Dateien gleich, nur größer. Füge noch zu jedem Restaurant eine erfundene Adresse noch hinzu und auch im Restaurant.js. Speichere die neuen Änderungen UND füge noch das Feature hinzu, dass der User einen Like oder Dislike geben kann.

Output:

- Detail-Panel funktioniert
- Adressen öffnen sich in Google Maps (war nicht gefragt)
- Bewertungssystem funktioniert